Microsoft System Design: Solutions

Reference solutions for the eight questions in the requirements cheat sheet. Each follows the interview script: APIs, data, architecture, deep dives, trade-offs. Compare against these after attempting a design cold, not before.

1. CDN

One-line answer: route each user to the nearest edge via DNS/anycast, serve from a tiered cache (edge → regional shield → origin), and invalidate through a global control plane.

APIs (control plane)

- `POST /distributions` {origin, domains, cache_rules} → distribution id
- `POST /distributions/{id}/purge` {paths} → 202 with purge id
- Data plane: plain HTTP `GET` on the customer's domain

Data

- Config store (strongly consistent, small): distributions, origins, cache rules, certificates. Replicated to every edge.
- Edge cache: key = host + path + selected query params/headers; value = response + headers + expiry. RAM for hot objects, SSD for the rest.

Architecture (request path)

1. DNS (GeoDNS or anycast) resolves to the nearest healthy point of presence (PoP).
2. PoP load balancer hashes the URL (consistent hashing) to one cache server, so each object lives once per PoP.
3. Hit → return. Miss → ask the regional shield cache.
4. Shield miss → fetch from origin, collapse duplicate requests into one, cache at shield and edge.

Deep dives

- **Request collapsing:** 1,000 simultaneous misses for the same object become one origin fetch.
- **Invalidation:** control plane fans purge messages to all PoPs through a pub/sub bus; versioned URLs (`app.v42.js`) avoid purges entirely.
- **Eviction:** LRU, with large rarely-hit objects evicted first.
- **Failure:** health checks remove a PoP from DNS/anycast; if origin is down, serve stale content (`stale-if-error`).
- **Security:** TLS terminated at the edge; DDoS absorbed across many PoPs; WAF rules at the edge.

Trade-offs to say out loud

- Longer TTLs mean higher hit ratio but staler content.
- Shield tier adds a hop but cuts origin load sharply.
- Anycast fails over fast; GeoDNS gives finer control but is slowed by DNS caching.

2. Content sharing via unique URL

One-line answer: clients upload directly to blob storage with a pre-signed URL; a metadata service assigns a short key; reads go through a CDN with access checks for private content.

APIs

- `POST /v1/content` {type, size, visibility, expires_at?} → {content_id, upload_url (pre-signed), short_url}
- `PUT <upload_url>` → client uploads bytes straight to blob storage
- `GET /{key}` → 302 to a CDN URL (public) or a short-lived signed URL (private, after auth); 404 / 410 expired
- `DELETE /v1/content/{key}` → 204 (owner only)

Data

- Metadata (key-value store, e.g. Cosmos DB / DynamoDB, sharded by key): `key, owner_id, blob_path, type, size, visibility, password_hash?, created_at, expires_at`
- Blobs in object storage (Azure Blob / S3), replicated across zones
- View counts streamed to an analytics store, not written to the metadata row

Architecture

1. Upload: client → API → metadata row (status=pending) + pre-signed URL → client uploads to blob → blob event marks status=ready (and triggers thumbnails or scanning).
2. Read: client → CDN → API resolves key (cache-aside in Redis) → redirect to the blob via CDN.

Deep dives

- **Key generation:** 7-char base62 from counter ranges (no collisions); use random 10+ char keys for private links so they can't be guessed.
- **Large files:** chunked, resumable uploads; never stream bytes through app servers.
- **Private content:** API checks permission, then issues a signed URL valid for minutes.
- **Expiry:** checked on read; background job deletes expired blobs.
- **Abuse:** rate limits per user, malware scan on upload.

Trade-offs to say out loud

- Pre-signed uploads offload bandwidth but make validation asynchronous.
- Sequential keys are short but guessable; random keys are safer but longer.

3. VM monitoring system

One-line answer: agents push batched metrics to a regional ingestion tier backed by a partitioned log (Kafka / Event Hubs); stream processors write to a time-series database and evaluate alert rules; queries hit the time-series store.

APIs

- `POST /v1/metrics` (agent) — batch of {vm_id, metric, timestamp, value, tags}
- `GET /v1/query?vm=...&metric=cpu&from=...&to=...&agg=p95&step=1m`
- `POST /v1/alerts` {expression, duration, severity, channels}

Data

- Time-series DB (e.g. Azure Data Explorer, InfluxDB, Prometheus-style TSDB): series key = metric + tags; points compressed (delta-of-delta timestamps, XOR values).
- Rollups: raw for 14 days, 1-min for 90 days, 1-hour for 1 year.
- Alert rules and dashboards in a small SQL store.

Architecture

1. Agent on VM batches every 10 s, compresses, retries with backoff; buffers locally if the backend is down.
2. Regional ingestion gateways authenticate and write to the log, partitioned by vm_id.
3. Consumers write to the TSDB (sharded by series key + time).
4. Stream processor evaluates alert rules on the live stream, keeps state per rule, deduplicates, then notifies.
5. Downsampling jobs build rollups; query service picks the right resolution for the time range.

Deep dives

- **Write volume (10M points/s):** batching, compression and the log absorb spikes; the TSDB is append-only.
- **Heartbeats:** missing data for N intervals fires a "VM down" alert; silence is itself a signal.
- **Cardinality:** limit tag values per tenant; high-cardinality tags (request IDs) explode storage.
- **Multi-region:** data stays in its region; a global query layer fans out.
- **Self-monitoring:** run the monitoring stack separately from what it monitors.

Trade-offs to say out loud

- Push vs pull: push works through firewalls and scales with agents; pull gives the server control over load.
- Stream alerting is fast; query-based alerting is simpler but adds delay.

4. E-commerce system

One-line answer: split into services by consistency need: catalog and search (eventually consistent, heavily cached), cart (fast key-value), and order/inventory/payment (strongly consistent, transactional), coordinated by a saga over a message bus.

APIs

- `GET /v1/products?q=...&cursor=...` · `GET /v1/products/{id}`
- `PUT /v1/cart/items` {product_id, qty}
- `POST /v1/orders` {cart_id, address_id, payment_method} + `Idempotency-Key` header → 201 {order_id, status}
- `GET /v1/orders/{id}`

Data

- Catalog: document store + search index (Elasticsearch / Azure AI Search), CDN for images.
- Cart: Redis or key-value store keyed by user_id, with TTL.
- Orders, inventory, payments: SQL (Postgres / Azure SQL), sharded by user_id or order_id.
- `inventory(sku, available, reserved)`; `orders(order_id, user_id, status, total, created_at)`.

Architecture (checkout saga)

1. Order service creates order (status=pending) using the idempotency key.
2. Inventory service reserves stock: `UPDATE ... SET available = available - qty WHERE available >= qty` (atomic, no oversell).
3. Payment service authorizes through the provider.
4. Success → order confirmed, event published for fulfillment and email. Failure → compensate: release stock, cancel order.

Deep dives

- **Oversell / flash sales:** atomic conditional decrements; for hot SKUs, pre-split stock into buckets or queue purchases.
- **Double charge:** idempotency keys end to end, including the payment provider call.
- **Reservation timeout:** reserved stock released if payment doesn't finish in ~10 min.
- **Traffic spikes:** autoscale stateless services, cache catalog aggressively, queue orders so checkout degrades instead of failing.
- **Search:** index updated asynchronously from catalog change events.

Trade-offs to say out loud

- Saga vs distributed transaction: saga avoids locking across services but needs compensation logic.
- Stale price or stock on product pages is acceptable; checkout re-validates.

5. On-prem to cloud migration

One-line answer: strangler-fig migration. Put a routing layer in front, move stateless services first, replicate data continuously into the cloud, shift traffic in small percentages with rollback at every step, then move the database primary last.

Plan in phases

1. **Assess:** inventory services, dependencies, data size, which parts are stateful; classify each as rehost, replatform or re-architect.
2. **Foundation:** cloud landing zone with networking (VPN / ExpressRoute to on-prem), identity, monitoring, infrastructure as code.

3. **Routing layer:** global load balancer or API gateway in front of both environments, so traffic can be split by percentage or by tenant.
4. **Stateless services:** containerize, deploy to the cloud, shift traffic 1% → 10% → 50% → 100% while comparing error rates and latency.
5. **Data:** initial bulk copy, then change data capture (CDC) to keep the cloud copy in sync; validate with row counts and checksums.
6. **Database cutover:** brief write freeze or dual-write window, promote the cloud database to primary, reverse replication back to on-prem as a rollback path.
7. **Decommission** on-prem after a soak period.

Deep dives

- **Zero downtime:** traffic shifting + CDC + reverse replication means every step can be undone.
- **Latency during hybrid:** keep chatty services and their data on the same side; migrate them together.
- **Consistency:** a single writer at any moment; avoid long dual-write periods (they drift).
- **Validation:** shadow traffic (mirror real requests to the cloud and compare responses) before taking live traffic.
- **Rollback triggers:** defined in advance by SLO (error rate, p99 latency).

Trade-offs to say out loud

- Lift-and-shift is fast but keeps old inefficiencies; re-architecting saves cost long term but adds risk and time.
- Migrating by tenant limits blast radius but runs both systems longer.

6. Message queue

One-line answer: topics split into partitions; each partition is an append-only log replicated across brokers with a leader; consumers in a group own partitions and commit offsets; at-least-once delivery with idempotent consumers.

Interfaces (Microsoft has asked for these)

```
 interface IProducer {
   Task<PublishResult> PublishAsync(string topic, Message msg, string? partitionKey = null);
}
interface IConsumer {
   Task<IReadOnlyList<Message>> ReceiveAsync(string topic, string group, int max, TimeSpan wait);
   Task AckAsync(string topic, string group, int partition, long offset);
}
record Message(string Id, byte[] Body, IDictionary<string,string> Headers, DateTime EnqueuedAt);
record PublishResult(string MessageId, int Partition, long Offset);
```

Data

- Each partition: segment files on disk + an offset index. Messages kept for a retention period (e.g. 7 days), not deleted on read.
- Metadata (topics, partition leaders, consumer group offsets) in a consensus store (ZooKeeper / Raft).

Architecture

1. Producer hashes partition key → partition → sends to that partition's leader broker.
2. Leader appends to its log, replicates to followers, acks once a quorum (e.g. 2 of 3) has it.
3. Consumer group coordinator assigns partitions to consumers; each consumer reads sequentially and commits its offset after processing.
4. Message fails N times → moved to a dead-letter topic.

Deep dives

- **Ordering:** guaranteed only within a partition; same key → same partition.
- **Delivery:** at-least-once (commit after processing); exactly-once needs idempotent producers plus transactional offset commits, or idempotent consumers.
- **Broker failure:** controller promotes an in-sync follower to leader.
- **Throughput:** sequential disk writes, batching, zero-copy reads; scale by adding partitions.

- **Rebalancing:** adding a consumer triggers partition reassignment; brief pause in consumption.

Trade-offs to say out loud

- Acking after quorum is safer but slower than acking on leader write.
- More partitions give more parallelism but weaker overall ordering and more metadata.

7. Real-time collaborative document editing

One-line answer: clients hold WebSocket connections to a collaboration server that owns each open document; edits are operations merged with OT (or CRDTs), appended to an operation log, broadcast to other editors, and periodically snapshotted.

APIs

- `POST /v1/docs` · `GET /v1/docs/{id}` (latest snapshot + version) · `POST /v1/docs/{id}/share`
- WebSocket `/v1/docs/{id}/live`: client sends `{op, base_version}`; server sends `{op, version, author}`, presence and cursor updates

Data

- Operation log (append-only, per document): `doc_id, version, op, user_id, timestamp`
- Snapshots every N ops or minutes, in blob storage; version history = snapshots + ops
- Doc metadata and permissions in SQL

Architecture

1. Client opens the doc: load the latest snapshot plus ops since, then connect the WebSocket.
2. Gateway routes all editors of one doc to the same collaboration server (consistent hashing on doc_id).
3. Client applies its edit locally at once, sends the op with its base version.
4. Server transforms the op against ops it hasn't seen, assigns the next version, persists, broadcasts.
5. Presence (cursors) sent via the same channel but never persisted.

Deep dives

- **OT vs CRDT:** OT needs a central server to order ops (simpler for server-based apps; Google Docs uses it). CRDTs merge without a central order (better for offline and peer-to-peer) but carry more metadata.
- **Server failure:** a new server takes over the doc by replaying the op log from the last snapshot; clients reconnect and resend unacknowledged ops.
- **Offline:** client queues ops, transforms them against missed ops on reconnect.
- **Spreadsheets:** cell-level ops are simpler than text, but formula dependencies need a recalculation graph.
- **Scale:** millions of docs, few editors each, so shard by doc_id.

Trade-offs to say out loud

- One server per doc makes ordering easy but creates a hot spot for very popular docs.
- Frequent snapshots load faster but cost more storage.

8. Cloud file storage (OneDrive-style)

One-line answer: split files into content-addressed chunks stored in blob storage; a metadata service tracks files, versions and which chunks make each version; a sync service notifies devices of changes so they download only the chunks that changed.

APIs

- `POST /v1/files/upload-session` {path, size, chunk_hashes[]} → which chunks are missing + upload URLs
- `PUT <chunk_url>` → upload each missing chunk (resumable)
- `POST /v1/files/commit` {session_id, base_version} → new version, or 409 on conflict
- `GET /v1/changes?cursor=...` → changes since the device's last cursor (plus long-poll or push notification)

- `GET /v1/files/{id}?version=...` → chunk list + download URLs

Data

- Metadata (SQL, sharded by user_id): `files(file_id, owner_id, path, latest_version)`, `versions(file_id, version, chunk_list, size, created_at)`, `shares(...)`, plus a per-user change journal for sync cursors
- Chunks (~4 MB) in blob storage keyed by SHA-256 hash, so identical chunks are stored once

Architecture

1. Client splits the file into chunks, hashes them, asks which are new.
2. Uploads only missing chunks directly to blob storage.
3. Commits the new version in metadata (conditional on base_version).
4. Change journal records it; notification service pushes to the user's other devices.
5. Other devices fetch changes by cursor, download only new chunks, reassemble.

Deep dives

- **Dedup and delta sync:** content hashes mean edits to a large file upload only changed chunks.
- **Conflicts:** commit fails on a stale base_version; the client saves a "conflicted copy" instead of overwriting.
- **Durability:** blob storage with zone or geo replication and erasure coding; never delete a chunk while any version references it (reference counting + garbage collection).
- **Hot metadata:** cache folder listings; shard by user so one user's tree stays together.
- **Sharing:** shared folders appear in each user's change journal via fan-out on write.

Trade-offs to say out loud

- Smaller chunks give better dedup but more metadata and requests.
- Cross-user dedup saves storage but complicates encryption and privacy.